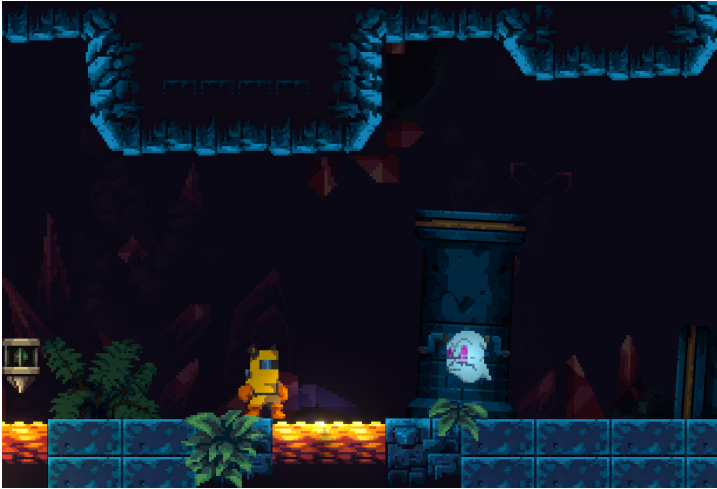


Projet en classe : La caverne de la mort

Dans les prochains cours (16 à 23), nous allons réaliser un jeu de type “platformer” où le joueur contrôle un personnage qui doit traverser un niveau et éviter des ennemis et des obstacles dangereux. Voici un **exemple jouable** de ça qu’on va créer en classe.



{ width : 80%; }

1.1 Cours 17 - Déplacer le jouer avec la physique

1. Ajouter un **Rigidbody2D** et un **BoxCollider2D** à l'objet Personnage.
2. Dans `Start()`, on récupère une référence `rb` au **Rigidbody2D** avec `GetComponent<>()`.
3. Créer un nouveau script `DeplacementPlatformer.cs`.

1.1.1 Configuration de contrôle

1. Importer la bibliothèque `UnityEngine.InputSystem`.
2. Définir deux variables publiques `InputAction` nommées `entreeMarche` et `entreeSaut`.
3. On prépare les `InputActions` dans `OnEnable()` et `OnDisable()`.
4. Dans l'**Inspector**, on ajoute des bindings : un positif/négatif pour `actionMarche` et un simple pour `actionSaut`.
5. Pour le mouvement et l'input, on va synchroniser les inputs avec la physique. Pour faire ça, on va dans **Edit > Project-Settings > Input System Package > Settings** et on change **Update Mode > Process Events in Fixed Update**.
6. Dans notre script, on va créer une fonction `FixedUpdate()` et, dans cette fonction, deux variables `float` locales : `axeMarche` avec la valeur de notre `actionMarche` et `intensiteMarche` avec la valeur absolue de cet axe (`Mathf.Abs(axeMarche)`).

1.1.2 Appliquer la logique de la marche

On veut avoir du mouvement de marche avec des accélérations pour donner un style plus fluide aux déplacements.

1. Créer des champs publics float pour `accclerationMarche`, `ralentissementMarche`, `vitesseXMax` et `vitesseXActuelle`.
2. Pour ralentir la marche :
3. Si on n'as pas d'intensité de marche (`intensiteMarche <= 0.01f`), on ralentisse la vitesse X du rb avec une interpolation vers 0f (`Mathf.Lerp(rb.linearVelocityX, 0, ralentissementMarche * Time.fixedDeltaTime)`).
4. On multiplie l'axeMarche par l'accclerationMarche et on somme au `rb.linearVelocityX`.
5. On limite la valeur de `rb.linearVelocityX` avec un `Mathf.Clamp()` et la `vitesseXMax`.

1.1.3 Appliquer la logique du saut

Pour le saut, on veut qu'il soit activé seulement quand le personnage est au sol. On veut aussi que l'impulsion du saut a une durée maximale : ça nous permet d'avoir un saut d'hauteur variable selon le temps qu'on enfonce la touche, en lieu d'une hauteur toujours égale.

1. Créer des champs publics float pour `accclerationSaut`, `tempsSaut`, `dureeImpulsionSaut`, `vitesseYMax`, un champs LayerMask `calquesSol` et un champs bool `estAuSol`.
2. Pour faire la **détection du sol** :
 1. On va tester si le personnage est au sol avec la méthode `Physics2D.Raycast(rb.position, Vector2.down, 1f, calquesSol)` qui lance un rayon d'un mètre de longueur à partir de son pivot vers en bas. Les objets hors les `calquesSol` vont être ignorés.
 2. On garde le résultat dans une variable locale `RaycastHit2D hit`.
 1. Si `hit.collider` est nul, le rayon n'a pas touché d'objets dans les calques du sol.
 2. Si `hit.collider` n'est pas nul, le rayon a touché le sol.
 3. On garde le résultat de ces conditions dans la variable `estAuSol`.
3. Pour créer la **hauteur variable**, on va créer trois ifs pour traiter notre bouton :
 1. Si `actionSaut.WasPressedThisFrame()` && `estAuSol`, on va commencer le saut, donc le `tempsSaut = 0f`.
 2. Si `actionSaut.IsPressed()`, le bouton est actif dans ce cadre.
 1. Si le `tempsSaut < dureeImpulsionSaut`, on somme l'impulsionSaut avec la vitesse Y du rb.
 2. On augmente le `tempsSaut` pour montrer que plus de temps enfoncé c'est passé.
 3. Si `actionSaut.WasReleasedThisFrame()`, le bouton est relâché, donc on change le `tempsSaut` à l'infini pour éviter que l'impulsion soit appliquée.

1.1.4 Limiter la vitesse de mouvement

Avec les déplacements physiques, c'est toujours intéressant de placer des limites minimales et maximales pour éviter des glitches ou des valeurs extrêmes.

1. On utilise la méthode `Mathf.Clamp()` et nos variables `vitesseXMax` et `vitesseYMax` pour limiter la valeur de `rb.linearVelocityX` et `rb.linearVelocityY`.
2. On garde la valeur de `rb.linearVelocityX` dans la variable `vitesseXActuelle` parce qu'elle sera utile pour gérer les animations et le visuel du personnage.

1.2 Cours 18 - Contrôle de caméra avec Cinemachine

À venir.

1.3 Cours 19 - Gestion d'animations multiples avec Animator pt.1

À venir.

1.4 Cours 20 - Gestion d'animations multiples avec Animator pt.2

À venir.

1.5 Cours 21 - Instanciation de projectiles

À venir.

1.6 Cours 22 - Gestion d'ennemis et listes

À venir.

1.7 Cours 23 - Post processing et lumières 2D

À venir.